

MATRICES

Dans TP n°6 consacré aux matrices, nous décrivons la syntaxe de la structure de données de type tableau et nous apprenons à la manipuler.

Un tableau à deux dimensions n et m , que nous appellerons aussi matrice $n \times m$, est un assemblage de n vecteurs de même dimension m , c'est-à-dire une suite finie à deux indices constituée d'éléments de même nature.

Dans cette structure de données, les éléments sont désignés par un couple de coordonnées entières, que l'on appelle couple d'indice. Le premier indice correspond au numéro de la ligne et le second désigne celui de la colonne.

Signalons que pour SCILAB, tout est de type tableau : un nombre est un tableau 1×1 , un vecteur ligne est un tableau $1 \times n$, un vecteur colonne est un tableau $n \times 1$, une chaîne de caractère est un vecteur dont les coordonnées correspondent aux codes ASCII des caractères de la chaîne, . . .

1 Découverte des matrices :

Les matrices sont la généralisation des vecteurs.

Il s'agit d'un tableau de nombres arrangés sous la forme de n lignes et m colonnes.

On parle alors d'une matrice $n \times m$.

Exemple :

$M = \begin{bmatrix} 5 & 6 & 8 & \pi & 8 & 1 & 2 & -2 \end{bmatrix}$ est une matrice à deux lignes et 4 colonnes (2×4).

Un vecteur est donc une matrice à une seule ligne ($1 \times n$) et un scalaire est une matrice (1×1).

L'appel à une valeur se fait à l'aide de deux indices qui désigne **la ligne puis la colonne** : **Mat(n°ligne,n°colonne)**.

Remarques : Il existe des matrices à n dimensions avec n indices.

Une matrice est écrite entre crochets.

```
1. Taper : >> M = [3 8 6; 2 5 1; 8 8 2; 1 3 4] // ";" est équivalent à un retour à la ligne
>> M = [3 8 6;
>>      2 5 1;
>>      8 8 2;
>>      1 3 4] // est plus long, mais peut-être plus lisible
>> M(1,2) // SCILAB renvoie la composante de la 1ère ligne et de la 2ème colonne (8)
>> M(4,3) // SCILAB renvoie la composante de la 4ème ligne et de la 3ème colonne (4)
>> M(3,4) // SCILAB vous indique que l'indice dépasse les dimensions de la matrice, index invalide.
```

2. Il est également possible de saisir les valeurs une à une.

Taper :

```
>> A(1,1) = 2
>> A(1,2) = 5 // on constate que la matrice s'agrandit progressivement.
>> A(3,3) = 3 // les cases non définies sont créées avec la valeur 0.
```

3. Taper une fonction ($A = \text{multiplication}(n,m)$) qui produise une table de multiplication ($i \times j$ pour i et j allant de 1 à n et m).

4. Il existe des fonctions très utiles qui créent des matrices de taille $n \times m$ avec des valeurs particulières.

Taper et commenter :

```
>> M = ones(2,3)
>> N = zeros(3,4)
>> O = rand(4,5)
>> P = eye(4,4)
```

5. La fonction **size** donne les dimensions de la matrice.

Taper :

```
>> [n,m]=size(M) // permet de stocker les dimensions de M dans les variables n et m.
>> size (M,1) et size (M,2) // représentent respectivement le nombre de lignes et le nombre de colonnes
>> Écrire une fonction maxmat, dont la variable est une matrice A, qui détermine le plus grand des éléments de A. En fait, cette fonction existe déjà dans SCILAB et s'appelle max mais, là encore, il est intéressant de savoir écrire le script.
>> length(M) // length renvoie le nombre d'éléments de M
```

6. Pour récupérer plusieurs valeurs d'une matrice, il suffit de remplacer les indices par des vecteurs.

Taper :

```
>> V = O(2,[1 3]) // on crée le vecteur V avec les colonnes 1 et 3 de la 2ème ligne de O
>> A = O([1 :3],[3 5]) // on crée la matrice A avec les colonnes 3 et 5 des lignes 1, 2 et 3
>> B = O(2 : $, 2) // on crée la matrice B avec la 2ème colonne des lignes 2, 3 et 4
```

7. Si l'ensemble des lignes nous intéresse, il suffit de remplacer l'indice des lignes par ' : '.

Taper :

```
>> A( :, 2) // on sélectionne toutes les lignes de la 2ème colonne
>> A(3, :) // on sélectionne toutes les colonnes de la 3ème ligne
>> M(3, :) = 5 // on affecte la valeur 5 à toutes les colonnes de la 3ème ligne
```

8. Exercice.

Nettoyer la mémoire ($>> \text{clear}$) puis créer une matrice A de 4×4 ne contenant que des 1 et une matrice B de 5×4 avec des coefficients quelconques.

Remplacer la 2^{ème} colonne de B par 3, 2, 5, 4, -9.

Remplacer la 4^{ème} colonne de A par des zéros.

Remplacer la 3^{ème} ligne de A par la 2^{ème} ligne de B.

Effectuer $>> A(3, 1 : 3) = A(3, 2 : 4)$. Que s'est-il passé ?

2 Concaténation des matrices :

9. Pour concaténer des matrices il suffit d'utiliser les crochets.

Taper :

```
>> A = ones(2,3), B = rand(2,4)
>> C = [A,B] // SCILAB réalise l'association (concaténation des matrices A et B)
>> A = ones(2,3), D = rand(3)
>> E = [A; D] // cette fois-ci la concaténation se fait à la verticale (grâce au ;)
```

Que donne >> [A,D] ?

On peut également raccourcir une matrice en ne sélectionnant qu'une partie de ses éléments. Taper :

```
>> E = E([2 :4],[1 3]) // on ne garde que les colonnes 1 et 3 des lignes 2 à 4
```

3 Opérations sur les matrices :

Nous allons distinguer deux types d'opérations sur les tableaux : les opérations élément par élément (que l'on appelle aussi opérations scalaires) et les opérations matricielles. SCILAB est capable de réaliser de très nombreuses opérations sur les matrices. Nous ne verrons en première année que les opérations de base.

10. Il est très courant d'utiliser les matrices comme outil de stockage de l'information et non comme un outil mathématique. Dans ce cas, il est très courant d'exécuter des **opérations sur chaque coefficient de la matrice**.

Pour cela il suffit de faire précéder l'opérateur désiré ($\times$, $/$, $\wedge$) d'un point.

Taper : >> M = [2 3; 4 5] et commenter la différence entre >> M×M et >> M.×M.

Faire de même pour >> M^3 et >> M.^3.

Y a-t-il une différence entre >> 3×M et >> 3.×M?

entre >> M + N et >> M .+ N?

11. Les fonctions usuelles que connaît SCILAB offrent la particularité de s'appliquer aussi bien à des éléments (en général des scalaires) qu'à des tableaux constitués de ces éléments. Tapez les expressions suivantes, suivies de la touche entrée :

```
>> M = [pi/2, pi; 0, 1],
>> exp(M).
```

12. L'addition et la soustraction matricielles n'ont pas à être (re)définies puisqu'elles coïncident avec les opérations élément par élément correspondantes. Pour effectuer $A * B$, il est nécessaire que les matrices A et B soient compatibles pour ce calcul, c'est-à-dire que le nombre de colonnes de A soit aussi le nombre de lignes de B. Tapez les expressions suivantes, suivies de la touche entrée :

```
>> M=[-9, 7, 3; -13, 10, 4; 4, -3, -1], N=[1, 0; 2, 0; 3, 1]
```

```
>> M * N, M^3, N * M
```

Remarque :

La puissance matricielle ne s'applique qu'à des matrices carrées!!!

13. En dehors des opérations arithmétiques introduites ci-dessus, il existe autre opération matricielle que nous introduirons. Du point de vue syntaxique, **la transposition** d'une matrice réelle A s'effectue en postfixant **une apostrophe** ', c'est-à-dire en écrivant A' (on notera la différence avec la notation mathématique qui consiste à préfixer l'exposant t).

Tapez les expressions suivantes, suivies de la touche entrée :

```
>> P = [1, 2; 3, 4; 5, 6]
```

```
>> P'
```

Il existe deux autres commandes dans le programme : ' **rank** ' et ' **inv** ' .

Taper et commenter :

```
>> rank(P)
```

```
>> inv(P)
```

4 Exercices

13.

- Écrire une fonction **matnulle** qui permette de construire la matrice nulle de taille $n \times m$. (en fait cette fonction existe déjà c'est)
- Écrire une fonction **matid** qui permette de construire la matrice identité In . (en fait cette fonction existe déjà c'est)

14.

- Écrire une fonction **matprod** qui effectue le produit de deux matrices lorsque c'est possible et vous signale l'erreur dans le cas contraire.
- Écrire une fonction **mattrace** qui calcule la trace d'une matrice carrée, c'est-à-dire la somme des coefficients diagonaux de cette matrice.

15.

- Écrire une fonction **danstab** qui teste si un élément x appartient à une matrice A.
- Écrire une fonction **permuteligne** qui permute deux lignes d'une matrice.

16. Écrire une fonction **pascal** qui récite les premières lignes du triangle de Pascal.

5 Ce que l'on retient :

- l'indexation des matrices M(ligne,colonne), l'utilisation de : et de \$,
- la fonction **size**,
- les fonctions **zeros**, **ones** pour créer des matrices avec des valeurs prédéfinies,
- la **concaténation** grâce aux crochets,
- les opérations sur les coefficients ($\times$, $\wedge$, $\div$) des matrices.

Il ne faut donc surtout pas oublier le . qui précède *, / ou ^ si l'on veut effectuer une multiplication, une division ou une élévation à la puissance TERME À TERME.